School of Engineering

HANA-bewegung eines geladenen Teilchens im Magnetfeld

Samuel Bortolato* Michael Saxer†

Dezember 2024

*bortosam@students.zhaw.ch
†saxermi1@students.zhaw.ch

Abstract

In diesem Projekt wird die Bewegung geladener Teilchen im Magnetfeld untersucht. Dabei werden die Maxwell-Gleichungen zur Modellierung statischer Magnetfelder herangezogen und numerisch mit der Finite-Elemente-Methode gelöst. Für die Analyse wurden sowohl detaillierte als auch homogenisierte Spulengeometrien betrachtet, um die Berechnung von Stromdichten und magnetischen Feldern zu ermöglichen. Zusätzlich wird die Bahn geladener Teilchen mithilfe eines gekoppelten Anfangswertproblems simuliert, wobei das explizite Runge-Kutta-Verfahren 4. Ordnung zum Einsatz kam. Außerdem wird ein physik-informiertes neuronales Netzwerk (PINN) implementiert, das die Lorentzkraftgleichung löst. Durch die Einbettung physikalischer Gesetze in die Loss-Funktion konnte das PINN die Teilchenbewegung approximieren, ohne direkt auf umfangreiche Datensätze angewiesen zu sein. Die Ergebnisse zeigen jedoch, dass mit unserem Ansatz keine saubere Lösung erzielt werden konnte. Abschließend schlagen wir mögliche Ansätze vor, um die PINN-Performance zu verbessern.

Inhaltsverzeichnis

1 Einführung **1**

2 Theoretische Grundlagen **2**

2.1 Spule mit wenige Windungen . . . 2
2.2 Spule mit vielen Windungen . . . 4
2.3 Geladenes Teilchen im Magnetfeld . . . 6
2.4 Schwache Gleichung des elektrischen Potentials φ . . . 7
2.5 Schwache Gleichung des Magnetisches Potential A . . . 8

3 Spulengeometrien **9**

4 Berechnung des magnetischen Vektorpotentials und der magnetischen Flussdichte **10**

4.1 Magnetisches Vektorpotential A . . . 10
4.2 Magnetische Flussdichte B . . . 12
4.3 Vergleich . . . 13

5 Partikelverhalten im Magnetfeld **14**

5.1 Abgestossenes Elektron . . . 14
5.2 Proton auf Kreisbahn . . . 17

6 Implementierung und Training eines Physik-informierten Neuronalen Netzwerks (PINN) **20**

6.0.1 Problemstellung . . . 20
6.0.2 Warum PINNs? . . . 20
6.1 Netzwerkarchitektur . . . 21
6.1.1 Wahl der Architektur . . . 21
6.1.2 Anpassungen für physikalische Randbedingungen . . . 21
6.2 Design der Loss-Funktion . . . 21
6.2.1 Residuum aus den grundlegenden Bewegungsgleichungen . . . 21
6.2.2 Anfangsbedingungs-Loss . . . 22
6.2.3 Gesamtloss . . . 22
6.3 Datenaufbereitung und Normalisierung . . . 22
6.3.1 Feldinterpolation auf Basis eines Meshs . . . 22
6.3.2 Grenzen und Clamping des Definitionsbereichs . . . 23
6.4 Trainingsprozess . . . 23
6.4.1 Wahl des Optimierungsverfahrens . . . 23
6.4.2 Lernraten-Schedules . . . 23
6.4.3 Konvergenzkriterien . . . 23
6.5 Lernkurve . . . 23
6.6 Elektron Trajektorie . . . 25
6.7 Validierung und Test . . . 26
6.8 Mögliche Erweiterungen . . . 26

6.9 Fazit 26

7 Diskussion **27**

1 Einführung

In diesem Projekt wird das statische Magnetfeld einer Spule und die Bewegung geladener Teilchen unter dem Einfluss dieses Feldes analysiert. Ziel ist es, die physikalischen und mathematischen Grundlagen zu untersuchen und diese mithilfe numerischer Methoden, wie der Finite-Elemente-Methode und dem Runge-Kutta-Verfahren, anzuwenden.

Die Betrachtung umfasst die Modellierung von Spulen mit unterschiedlichen Windungszahlen sowie die Berechnung der zugehörigen Stromdichten und Magnetfelder. Zusätzlich werden die Bahnen geladener Teilchen im Magnetfeld simuliert.

Dieser Bericht stellt die theoretischen Grundlagen, die Implementierungsschritte und die wichtigsten Resultate vor. Besonderes Augenmerk liegt auf der geometrischen Vereinfachung von Spulen und der numerischen Lösung des gekoppelten Anfangswertproblems [1]. Der Code für dieses Projekt ist auf GitHub verfügbar [2].

2 Theoretische Grundlagen

Die folgenden Abschnitte liefern das physikalische und mathematische Fundament für die spätere Modellierung, Simulation und Berechnung des statischen Magnetfeldes im dreidimensionalen Raum einer Spule.

Grundlegend für das Projekt ist die Maxwell Gleichung die wie folgt definiert werden kann:

$$\operatorname{curl} H = j_c, \quad \text{für} \quad x \in \Omega, \tag{1}$$
$$\operatorname{div} B = 0, \quad \text{für} \quad x \in \Omega, \tag{2}$$

wobei der Zusammenhang zwischen Magnetfeld H und der magnetischen Flussdichte B durch das Materialgesetz:

$$B = \mu H, \tag{3}$$

gegeben ist. Mit $\mu = \mu_0 \cdot \mu_r$, wobei $\mu_0 = 4\pi 10^{-7}$ [Vs/Am] die magnetische Feldkonstante und μ_r die relative Permeabilität ist. Für das ausführen des Projekts wird davon ausgegangen dass für die untersuchte Geometrie $\mu_1 \equiv 1$ gilt.

2.1 Spule mit wenige Windungen

Aus der Maxwell Gleichung, das Induktionsgesetz besagt, dass:

$$\operatorname{curl} E = \nabla \times E = -\frac{\partial B}{\partial t}. \tag{4}$$

Da das untersuchte Elektrische Feld E aber statisch ist, und somit wirbelfrei, gilt:

$$\operatorname{curl} E = 0. \tag{5}$$

Über:

$$\operatorname{curl}(\nabla\varphi(x)) = \nabla \times \nabla\varphi(x) = \begin{pmatrix} \frac{\partial}{\partial x} \\ \frac{\partial}{\partial y} \\ \frac{\partial}{\partial z} \end{pmatrix} \times \begin{pmatrix} \frac{\partial \varphi(x)}{\partial x} \\ \frac{\partial \varphi(x)}{\partial y} \\ \frac{\partial \varphi(x)}{\partial z} \end{pmatrix} = \begin{pmatrix} \frac{\partial^2 \varphi(x)}{\partial y \partial z} - \frac{\partial^2 \varphi(x)}{\partial z \partial y} \\ \frac{\partial^2 \varphi(x)}{\partial z \partial x} - \frac{\partial^2 \varphi(x)}{\partial x \partial z} \\ \frac{\partial^2 \varphi(x)}{\partial x \partial y} - \frac{\partial^2 \varphi(x)}{\partial y \partial x} \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \\ 0 \end{pmatrix}$$

wird bewiesen dass für jedes zweimal stetig differenzierbares Skalarfeld $\nabla\varphi(x)$ die Identität:

$$\operatorname{curl} \nabla\varphi(x) = 0 \tag{6}$$

gilt.

Für ein Gradientenfeld wird daher:

$$E = -\nabla\varphi, \tag{7}$$

gesetzt, wobei $\varphi(x)$ das Skalarfeld des elektrische Potentials ist.

Aus der Ladungserhaltung ergibt sich:

$$\operatorname{div} j_c = 0, \quad \text{mit} \quad j_c = \sigma E, \tag{8}$$

wobei σ die Leitfähigkeit des Materials beschreibt.

In diesem Projekt wird die Stromdichte auf Γ_{in} definiert, sodass folgendes Randwertproblem entsteht:

$$-\operatorname{div}(\sigma\nabla\varphi) = 0, \quad \text{für} \quad x \in \Omega_{\text{coil}} \tag{9}$$

$$\partial_n \vec{j_c} = \frac{i_0}{A_{\text{Windung}}}, \text{ auf } \quad \vec{x} \in \Gamma_{\text{in}} \tag{10}$$

$$\varphi = 0, \qquad \text{auf} \quad \vec{x} \in \Gamma_{\text{out}} \tag{11}$$

$$\partial_n \vec{j_c} = 0, \qquad \text{auf} \quad \vec{x} \in \Gamma_{\text{bnd}} \tag{12}$$

Die ersten beiden Randbedingungen definieren die einfliessende Stromdichte sowie ein Bezugs-Potential, um die Eindeutigkeit des Problems sicherzustellen. Die letzte Bedingung beschreibt, dass über den restlichen Rand der Spulenwindungen kein Strom nach aussen fliesst.

Für das Magnetfeld B wird das magnetische Vektorpotential A eingesetzt:

$$B = \operatorname{curl} A. \tag{13}$$

Mit:

$$\operatorname{div}(\operatorname{curl}(A(x))) = \begin{pmatrix} \frac{\partial A_3}{\partial y} - \frac{\partial A_2}{\partial z} \\ \frac{\partial A_1}{\partial z} - \frac{\partial A_3}{\partial x} \\ \frac{\partial A_2}{\partial x} - \frac{\partial A_1}{\partial y} \end{pmatrix} \cdot \begin{pmatrix} \frac{\partial}{\partial x} \\ \frac{\partial}{\partial y} \\ \frac{\partial}{\partial z} \end{pmatrix} = \begin{pmatrix} \frac{\partial}{\partial x}\left(\frac{\partial A_3}{\partial y} - \frac{\partial A_2}{\partial z}\right) \\ \frac{\partial}{\partial y}\left(\frac{\partial A_1}{\partial z} - \frac{\partial A_3}{\partial x}\right) \\ \frac{\partial}{\partial z}\left(\frac{\partial A_2}{\partial x} - \frac{\partial A_1}{\partial y}\right) \end{pmatrix} = \begin{pmatrix} \frac{\partial^2 A_3}{\partial x \partial y} - \frac{\partial^2 A_2}{\partial x \partial z} \\ \frac{\partial^2 A_1}{\partial y \partial z} - \frac{\partial^2 A_3}{\partial y \partial x} \\ \frac{\partial^2 A_2}{\partial z \partial x} - \frac{\partial^2 A_1}{\partial z \partial y} \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \\ 0 \end{pmatrix}$$

Es wird bewiesen, dass für jedes zweimal stetig differenzierbares Vektorfeld A:

$$\operatorname{div}\operatorname{curl} A = \nabla \cdot (\nabla \times A) = 0 \tag{14}$$

gilt und somit wird (2) weiterhin erhalten.

Die verbleibende Maxwell-Gleichung (1) wird mit dem Vektorpotential A somit in die folgende Form gebracht:

$$\operatorname{curl}(\mu^{-1}\operatorname{curl} A) = j_c \tag{15}$$

Diese entspricht der starken Form des magnetischen Potenzials.

2.2 Spule mit vielen Windungen

Die Modellierung einer Spule mit vielen Windungen erhält zwei Probleme: Erstens ist die Auflösung der Windungen als Mesh entweder nicht möglich oder sehr aufwendig. Zweitens wäre das Problem, selbst wenn eine Auflösung möglich wäre, schlecht konditioniert und würde einen grossen Rechenaufwand erfordern. Folglich wird die Geometrie vereinfacht, und die Stromdichte der homogenisierten Spule wird über die folgende analytische Formel berechnet:

$$j_c(x) = N_c \cdot \frac{i_c}{A_c} \begin{pmatrix} y/r \\ -x/r \\ 0 \end{pmatrix}, \quad r = \sqrt{x^2 + y^2 + z^2}. \tag{16}$$

Wobei N_c die Anzahl der Windungen, i_c der Strom pro Windung und A_c die Querschnittsfläche des Windungspakets ist.

Die Stromdichte j_c, welche durch die Kupferwindungen fliesst, wird durch den respektiven Strom induziert. Grundlegend ist, dass j_c quellenfrei ist und somit:

$$\operatorname{div}(j_c) = 0 \tag{17}$$

erfüllt und somit quellenfrei ist.

Die Divergenz der Stromdichte (16) berechnet sich zu:

$$\operatorname{div}(j_c(x)) = k \left(\frac{\partial}{\partial x}\left(\frac{y}{r}\right) + \frac{\partial}{\partial y}\left(-\frac{x}{r}\right) + 0 \right). \tag{18}$$

Die Ableitungen von r lauten:

$$\frac{\partial r}{\partial x} = \frac{\partial}{\partial x}\sqrt{x^2 + y^2 + z^2} = \frac{x}{\sqrt{x^2 + y^2 + z^2}} = \frac{x}{r}, \tag{19}$$

$$\frac{\partial r}{\partial y} = \frac{\partial}{\partial y}\sqrt{x^2 + y^2 + z^2} = \frac{y}{\sqrt{x^2 + y^2 + z^2}} = \frac{y}{r}. \tag{20}$$

Die Quotientenregel zur Ableitung eines Bruchs lautet:

$$\frac{\partial}{\partial x}\left(\frac{f(x)}{g(x)}\right) = \frac{\frac{\partial f}{\partial x}\cdot g(x) - f(x)\cdot\frac{\partial g}{\partial x}}{g(x)^2}.$$

Dabei ist $f(x)$ der Zähler und $g(x)$ der Nenner des Bruchs. Wir wenden diese Regel auf die Komponenten der Divergenz an.

1. Erste Komponente von (18):

$$\frac{\partial}{\partial x}\left(\frac{y}{r}\right) = \frac{\frac{\partial y}{\partial x}\cdot r - y\cdot\frac{\partial r}{\partial x}}{r^2}.$$

Da y nicht von x abhängt, gilt $\frac{\partial y}{\partial x} = 0$. Somit bleibt:

$$\frac{\partial}{\partial x}\left(\frac{y}{r}\right) = \frac{0\cdot r - y\cdot\frac{\partial r}{\partial x}}{r^2}.$$

Einsetzen von $\frac{\partial r}{\partial x} = \frac{x}{r}$ aus (19) ergibt:

$$\frac{\partial}{\partial x}\left(\frac{y}{r}\right) = \frac{-y\cdot\frac{x}{r}}{r^2} = -\frac{xy}{r^3}.$$

Zweite Komponente von (18):

$$\frac{\partial}{\partial y}\left(-\frac{x}{r}\right) = \frac{-\frac{\partial x}{\partial y}\cdot r - \cdot(-x)\cdot\frac{\partial r}{\partial y}}{r^2}.$$

Da x nicht von y abhängt, gilt $\frac{\partial x}{\partial y} = 0$. Somit bleibt:

$$\frac{\partial}{\partial y}\left(-\frac{x}{r}\right) = -\frac{0\cdot r + x\cdot\frac{\partial r}{\partial y}}{r^2}.$$

Einsetzen von $\frac{\partial r}{\partial y} = \frac{y}{r}$ aus (20) ergibt:

$$\frac{\partial}{\partial y}\left(-\frac{x}{r}\right) = \frac{x\cdot\frac{y}{r}}{r^2} = \frac{xy}{r^3}.$$

Zusammengefasst lauten die drei Komponenten von (18):

$$\frac{\partial}{\partial x}\left(\frac{y}{r}\right) = -\frac{xy}{r^3}, \quad \frac{\partial}{\partial y}\left(-\frac{x}{r}\right) = \frac{xy}{r^3}, \text{und } 0.$$

Die Divergenz der Stromdichte ergibt sich zu:

$$\operatorname{div}(j_c(x)) = k\left(-\frac{xy}{r^3} + \frac{xy}{r^3} + 0\right) = k \cdot (0) = 0.$$

Die Divergenz der Stromdichte verschwindet, was die Ladungserhaltung (17) bestätigt.

2.3 Geladenes Teilchen im Magnetfeld

Ein geladenes Teilchen, das sich in einem Magnetfeld bewegt, erfährt eine Ablenkung durch die Lorentzkraft F, die sich folgendermassen ausdrückt:

$$F(x) = q\, v(t) \times B, \tag{21}$$

wobei q die Ladung des Teilchens und $v(t)$ der Geschwindigkeitsvektor des Teilchens ist. Die Bewegung des Teilchens wird durch die Newton'sche Bewegungsgleichung beschrieben:

$$\dot{s}(t) = v(t), \tag{22}$$
$$\ddot{s}(t) = a(t), \tag{23}$$

mit der Beschleunigung $a(t)$, die aus dem Kräftegleichgewicht folgt:

$$m\, a(t) = q\, v(t) \times B. \tag{24}$$

Daraus ergibt sich das gekoppelte Anfangswertproblem zur Berechnung der Bahn $s(t)$:

$$m\, \ddot{s}(t) = q\, v(t) \times B(s(t)), \tag{25}$$
$$s(0) = s_0, \tag{26}$$
$$v(0) = v_0. \tag{27}$$

Hierbei sind s_0 die Startposition und v_0 die Startgeschwindigkeit des Teilchens. Die Berechnung der Bahn erfordert die numerische Lösung dieses Anfangswertproblems, insbesondere wenn $B(s(t))$ räumlich inhomogen ist.

Zur numerischen Berechnung der Bahn wird ein explizites Verfahren, wie beispielsweise das Runge-Kutta-Verfahren 4. Ordnung, verwendet. Dies ermöglicht es, die Trajektorien von Elektronen oder Protonen im Magnetfeld mit beliebigen Anfangsbedingungen zu simulieren. Dabei hängt die Stabilität und Genauigkeit der Lösung von der Wahl der Zeitschrittweite ab.

Zusammengefasst beschreibt die Bewegung eines geladenen Teilchens im Magnetfeld eine Anwendung der Maxwell-Gleichungen auf ein gekoppelt-dynamisches System. Diese Theorie ergänzt die Berechnung

des Magnetfelds und liefert die Grundlage für weiterführende Simulationen im Projekt.

2.4 Schwache Gleichung des elektrischen Potentials φ

Die starke Form des Elektrische Potential (9) lautet:

$$-\text{div}(\sigma \nabla \varphi) = 0.$$

Multiplizieren mit einer Testfunktion ψ und integrieren über den Bereich Ω ergibt:

$$\int_\Omega -\text{div}(\sigma \nabla \varphi) \cdot \psi \, d\Omega = 0.$$

Anwendung des Divergenzsatzes, wobei $\vec{n}$ der Normalenvektor auf den Rand $\partial\Omega$ ist:

$$\int_\Omega -\text{div}(\sigma \nabla \varphi) \cdot \psi \, d\Omega = \int_{\partial\Omega} \sigma(\nabla \varphi \cdot \vec{n}) \cdot \psi \, \text{d}s - \int_\Omega \sigma \nabla \varphi \cdot \nabla \psi \, d\Omega.$$

Dies führt zu [1]:

$$\int_\Omega \sigma \nabla \varphi \cdot \nabla \psi \, d\Omega = \int_{\partial\Omega} \sigma(\nabla \varphi \cdot \vec{n}) \cdot \psi \, ds. \tag{28}$$

Randbedingungen

Die Randbedingungen werden nun berücksichtigt:

1. Auf Γ_{in} zufolge (10) gilt :

$$\int_{\Gamma_{\text{in}}} \sigma(\nabla \varphi \cdot \vec{n}) \cdot \psi \, \text{d}s = \int_{\Gamma_{\text{in}}} \frac{i_0}{A_{\text{Windung}}} \psi \, ds.$$

2. Auf Γ_{out} zufolge (11) gilt:

$$\int_{\Gamma_{\text{out}}} \sigma(\nabla \varphi \cdot \vec{n}) \cdot \psi \, \text{d}s = 0.$$

3. Auf Γ_{bnd} zufolge (12) gilt:

$$\int_{\Gamma_{\text{bnd}}} \sigma(\nabla \varphi \cdot \vec{n}) \cdot \psi \, \text{d}s = 0.$$

Zusammenfassung der Schwachen Form

Einsetzen der Randbedingungen in (28) ergibt die schwache Form:

$$\int_\Omega \sigma \nabla \varphi \cdot \nabla \psi \, d\Omega = \int_{\Gamma_{\text{in}}} \frac{i_0}{A_{\text{Windung}}} \psi \, \text{d}s. \tag{29}$$

Diese Gleichung beschreibt die schwache Form des Randwertproblems für das elektrische Potential φ unter den gegebenen Randbedingungen (10), (11) und (12).

2.5 Schwache Gleichung des Magnetisches Potential A

Die Gleichung für das magnetische Potential A (15) lautet:

$$\operatorname{curl}\left(\mu^{-1}\operatorname{curl} A\right) = j_c \tag{30}$$

Multiplizieren mit einer Testfunktion v und integrieren über den Bereich ergibt:

$$\int_\Omega \operatorname{curl}(\mu^{-1}\operatorname{curl} A)\cdot v\,\mathrm{d}x = \int_\Omega j_\mathrm{c}\cdot v\,\mathrm{d}x \tag{31}$$

Die partielle Integration der linke Seite ergibt:

$$\int_\Omega \operatorname{curl}(\mu^{-1}\operatorname{curl} A)\cdot v\,\mathrm{d}x = \frac{1}{\mu}\int_\Omega \operatorname{div}(\operatorname{curl} A\times v)\,\mathrm{d}x + \frac{1}{\mu}\int_\Omega \operatorname{curl} A\cdot\operatorname{curl} v\,\mathrm{d}x \tag{32}$$

Mit der Anwendung des Diverergetzsatzes ergibt sich von (31) mit (32):

$$\frac{1}{\mu}\int_{\partial\Omega}(\operatorname{curl} A\times v)\cdot n\,\mathrm{d}s + \frac{1}{\mu}\int_\Omega \operatorname{curl} A\cdot\operatorname{curl} v\,\mathrm{d}x = \int_\Omega j_\mathrm{c}\cdot v\,\mathrm{d}x \tag{33}$$

Werden die Nedelec Elemente, folglich die Drichlet Bedingung, für die Diskretisierung des Problems, eingesetzt, ergibt sich die schwache Gleichung:

$$\frac{1}{\mu}\int_\Omega \operatorname{curl} A\cdot\operatorname{curl} v\,\mathrm{d}x = \int_\Omega j_\mathrm{c}\cdot v\,\mathrm{d}x \tag{34}$$

Da die schwache Gleichung nicht eindeutig definiert ist, kann die Gleichung mit Hilfe der Massenmatrix regularisiert:

$$M = \int_\Omega u\cdot v\,\mathrm{d}x.$$

Das rezirkularisierte lineare Gleichungssystem lautet folglich:

$$\frac{1}{\mu}\int_\Omega \operatorname{curl} A\cdot\operatorname{curl} v\,\mathrm{d}x + k\int_\Omega A\cdot v\,\mathrm{d}x = \int_\Omega j_\mathrm{c}\cdot v\,\mathrm{d}x,\quad \forall v\in H(\mathrm{curl},\Omega). \tag{35}$$

Die numerische Lösung dieser Gleichung erfolgt in einem passenden Funktionsraum, dem sogenannten $H(\mathrm{curl},\Omega)$-Raum:

$$H(\mathrm{curl},\Omega) = \{v\in[L^2(\Omega)]^3 \mid \operatorname{curl} v\in[L^2(\Omega)]^3\}. \tag{36}$$

Dieser Raum ermöglicht es, die schwache Formulierung der Gleichungen numerisch zu lösen. In der Praxis wird eine Diskretisierung mit Nédelec-Elementen verwendet welche in `ngsolve` verfügbar sind.

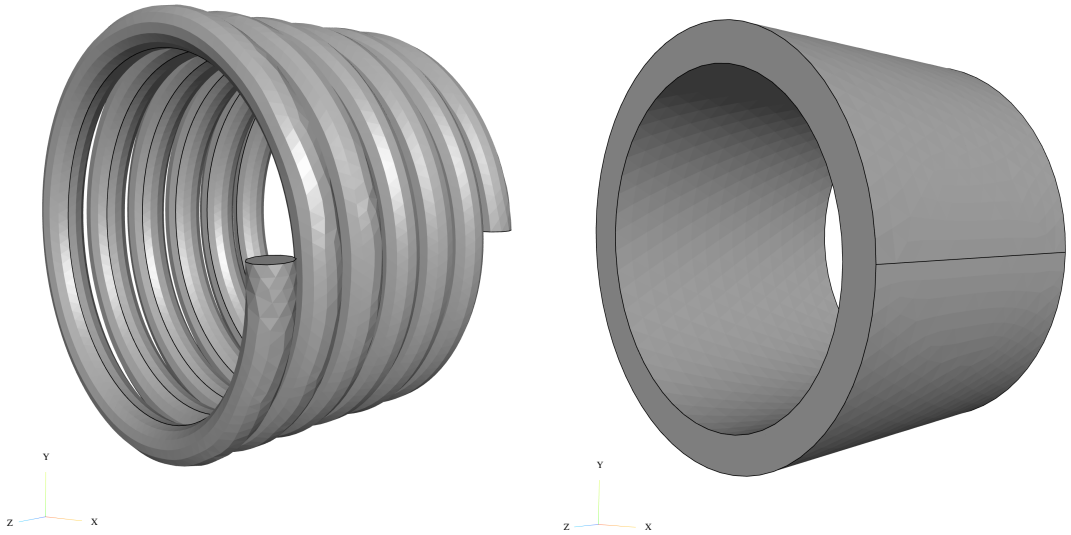
3 Spulengeometrien

Für die Simulation und Berechnung mit der Finite-Elemente-Methode (FEM), die in den folgenden Kapiteln untersucht werden, werden zwei Geometrien verwendet: eine detaillierte, realitätsnahe Spulengeometrie, die als inhomogene Geometrie betrachtet wird, sowie eine vereinfachte Geometrie in Form eines hohlen Zylinders.

Diese Geometrien wurden mithilfe der Listings 5 und 6 aus der Aufgabenstellung [2] generiert.

Im Detail wurden im Skript `coil_geometry.py` zwei Funktionen erstellt: `create_coil_geometry()` und `create_homo_geometry()`. Diese repräsentieren die inhomogene, beziehungsweise die homogene, Geometrie der Spule basierend auf gegebenen Parametern.

Die Ergebnisse der Funktionen sind in den Abbildungen 1a und 1b visualisiert. Beide Abbildungen basieren auf den Standardparametern.



(a) Inhomogene Geometrie (Windungen ersichtlich).

(b) Homogenisierte Geometrie (Windungen nicht ersichtlich).

Abbildung 1: Darstellung der Geometrien, erstellt mithilfe der Funktionen in `coil_geometry.py`.

4 Berechnung des magnetischen Vektorpotentials und der magnetischen Flussdichte

Mit den erworbenen Kenntnissen der vorherigen Kapitel kann die Berechnung und Untersuchung des magnetischen Vektorpotentials A und der magnetischen Flussdichte B in den Spulen vorgenommen werden. Ziel dieses Kapitels ist es, die Unterschiede zwischen der Berechnung in der homogenen und der inhomogenen Spule zu untersuchen.

4.1 Magnetisches Vektorpotential A

Um das magnetische Vektorpotential A zu berechnen, ist die erhaltene schwache Gleichung (34) beziehungsweise das regularisierte Gleichungssystem davon (Gleichung (35)) für beide Geometrien grundlegend:

$$\frac{1}{\mu}\int_\Omega \operatorname{curl} A \cdot \operatorname{curl} v \,\mathrm{d}x + k\int_\Omega A \cdot v \,\mathrm{d}x = \int_\Omega j_c \cdot v \,\mathrm{d}x.$$

Der grösste Unterschied zwischen den beiden Geometrien liegt in der Definition der Stromdichte j_c.

In der **homogene Geometrie** kann diese mittels der analytische Formel (16) definiert werden.

```
# Geometrische Parameter
A_c = nwindings*(2*wireradius)*1.2*2*wireradius # Höhe * Breite * 2
r = sqrt(x**2 + y**2) #Radius von coil

# Stromdichte j_c gemäss Aufgabenstellung
j_c = CoefficientFunction((
    (nwindings * i_c / A_c) * y / r,
    -(nwindings * i_c / A_c) * x / r,
    0
))
```

In der **inhomogene Geometrie** dagegen muss j_c über die Schwache Form des Elektrischen Potentials (29) zuerst berechnet werden.

```
# Raum Definieren
V_phi = H1(mesh,order = order, dirichlet='out',definedon=mesh.Materials('coil'))
...

# Bilinearform definieren
a_phi = BilinearForm(V_phi)
a_phi += sigma_coil * grad(phi) * grad(psi) * dx

# Linearform definieren
f_phi = LinearForm(V_phi)
f_phi += i_c / crosssection * psi *ds("in")
...

# Lösen für gfphi
```

```
15 with TaskManager():
16     gfphi.vec.data = a_phi.mat.Inverse(V_phi.FreeDofs()) * f_phi.vec
```

Folglich gemäss (8) definiert sich j_c:

```
1 # Elektrisches Feld und Stromdichte aus berechnen
2 E = -grad(gfphi)  # Elektrisches Feld
3 j_c = sigma_coil * E  # Stromdichte
```

Wie in Kapitel 2.5 erläutert, eignet sich für die Lösung des Problems der Raum $H(\text{curl},\Omega)$. Dieser wird sowohl im inhomogenen Skript, `4_MagPot_InhomoCoil.ipynb`, als auch im homogenen Skript, `4_MagPot_HomoCoil.ipynb`, unter Berücksichtigung der Randbedingungen folgendermassen (in Zeile 4) definiert:

```
1 # H(curl)-Raum definieren
2 order = 4
3 V = HCurl(mesh, order=order-1, nograds=True, dirichlet="outer")
4 u, v = V.TnT()
5 gfA = GridFunction(V)
```

Die Bilinearform

$$\frac{1}{\mu}\int_\Omega \text{curl } A \cdot \text{curl } v \, \mathrm{d}x + k\int_\Omega A \cdot v \, \mathrm{d}x$$

wird folgenderweise definiert:

```
1 # Bilinearform und Linearform
2 a = BilinearForm(V, symmetric=True)
3 a += (1/mu) * InnerProduct(curl(u), curl(v)) * dx
4 a += k * InnerProduct(u, v) * dx
```

Und die Linearform

$$\int_\Omega j_\mathrm{c} \cdot v \, \mathrm{d}x$$

der schwache Gleichung:

```
1 # Linearform: Stromdichte
2 f = LinearForm(V)
3 f += InnerProduct(j_c, v) * dx("coil")
```

Wichtig ist, dass die Stromdichte korrekterweise auf der Spule berechnet wird (`dx("coil")`), da diese nicht in der Luft fliesst. Danach kann das Gleichungssystem gelöst werden:

```
1 # Lösen des Gleichungssystems mit CG-Verfahren und Präkonditionierer (krylovspace.CGSolver)
2 inv = krylovspace.CGSolver(a.mat, pre.mat, tol=1e-12, printrates = True)
```

```
with TaskManager():
    gfA.vec.data = inv * f.vec
```

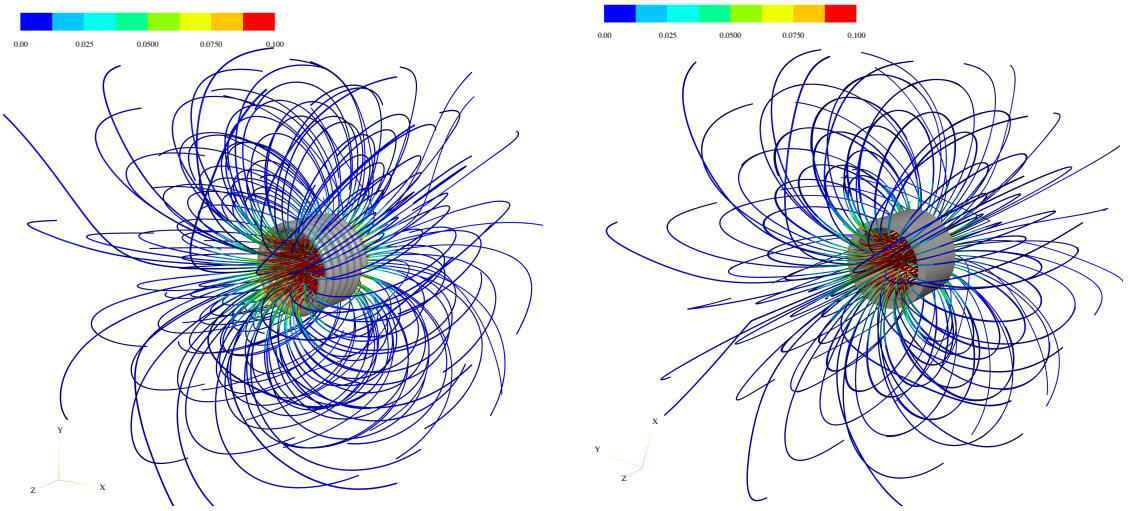
Um den Aufwand des Verfahrens zu reduzieren, wird ein Preconditioner benutzt, der mithilfe des `krylovespace.CGSolver()` das Gleichungssystem iterativ lösen kann.

4.2 Magnetische Flussdichte B

Die magnetische Stromdichte B ist gemäss (13) definiert:

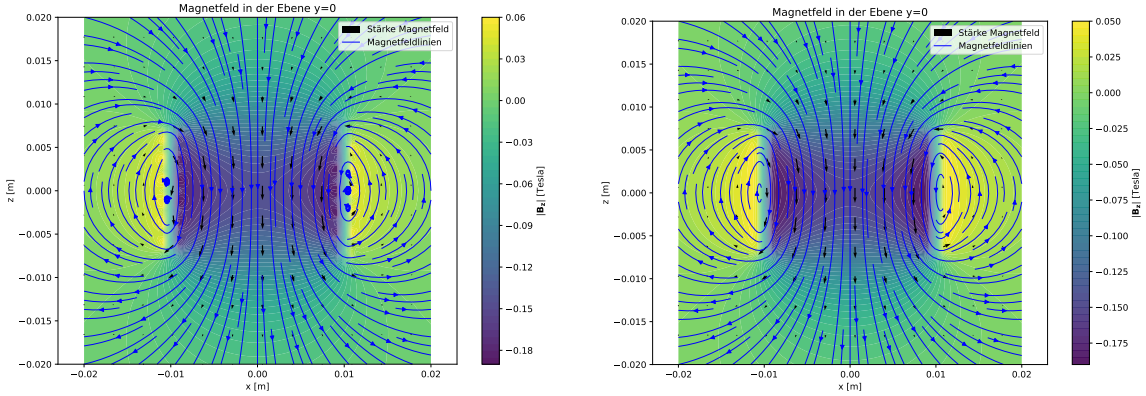
```
# Magnetfeld B aus Vekorpotential A
B = curl(gfA)
```

Aus dem Magnetfeld B können dann folglich die Feldlinien visualisiert werden. In den Abbildungen 2b und 2a ist das Magnetfeld B bei 500 A Strom in einer 3D-Darstellung visualisiert. In den Abbildungen 3b und 3a wird ein 2D-Plot auf der XZ-Ebene dargestellt.



(a) Magnetfeld bei Inhomogene Geometrie. (b) Magnetfeld bei Homogene Geometrie.

Abbildung 2: 3D Magnetfeld Visualisierung bei $i_c = 500$ A.



(a) Magnetfeld bei Inhomogene Geometrie. (b) Magnetfeld bei Homogene Geometrie.

Abbildung 3: 2D Magnetfeld Visualisierung bei $i_c = 500$ A in XZ-Ebene.

4.3 Vergleich

Eines der Ziele der Untersuchung und Berechnung des Magnetfeldes B ist der Vergleich zwischen der Berechnung der inhomogenen und homogenen Spule. Die Rechenzeiten bis zur Erzeugung des B-Feldes, ohne dessen Visualisierung, betragen:

Tabelle 1: Vergleich der mittleren Rechenzeiten zwischen inhomogenen und homogenen Spulen.

Metrik	**Inhomogene Spule (s)**	**Homogene Spule (s)**	**Differenz (s)**
Mittelwert	129.57	13.69	-115.88

Zusätzlich wurde auch die Abweichung zwischen den beiden Feldern untersucht. Aus dem homogenen Feld wurden 10000 randomisierte Punkte ausgewertet. Dieselben Punkte wurden auch im inhomogenen Feld ausgewertet. Der Unterschied zwischen dem inhomogenen und dem homogenen Feld beträgt

$$\mu \pm \sigma = -0.000215 \pm 0.001415\,\mathrm{T}$$

Die Verteilung kann folgendermassen visualisiert werden:

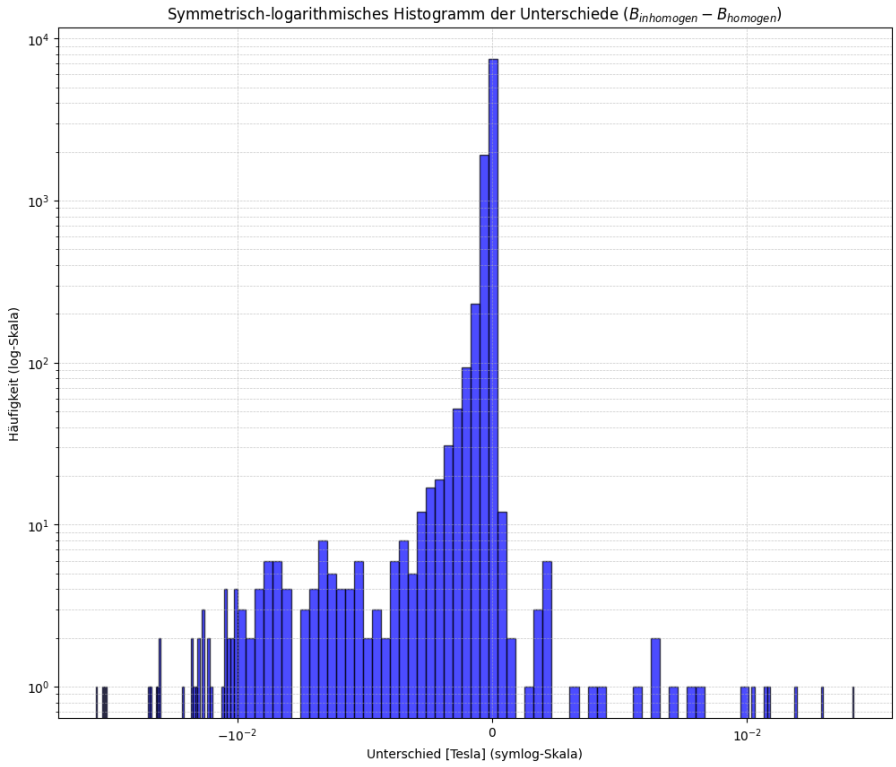


Abbildung 4: Histogram der Unterschiede $|B_{inhomogen} - B_{homogen}|$. Häufigkeit in log-Skala und Unterschied in symmetrische log-Skala.

5 Partikelverhalten im Magnetfeld

In diesem Abschnitt wird die numerische Berechnung der Bahn eines geladenen Teilchens im Magnetfeld der homogenisierten Spule betrachtet. Die Bewegung eines solchen Teilchens unterliegt einem gekoppelten Anfangswertproblem, das aus den Newton'schen Bewegungsgleichungen in Kombination mit der Lorentzkraft hergeleitet wird:

$$\vec{F}(x) = q \cdot \left(\vec{B} \times \vec{v}(t)\right) \tag{37}$$

$$m \cdot \ddot{\vec{s}}(t) = q \cdot \left(\vec{B}(\vec{s}(t)) \times \dot{\vec{s}}(t)\right), \tag{38}$$

$$\vec{s}(0) = \vec{s}_0, \tag{39}$$

$$\dot{\vec{s}}(0) = \vec{v}_0. \tag{40}$$

Um dieses Problem zu lösen, wird das explizite Runge-Kutta-Verfahren vierter Ordnung (RKV) eingesetzt. Dieses Verfahren wird aufgrund seiner hohen Genauigkeit und Einfachheit in der Implementierung gewählt. Mathematisch lässt sich das RKV durch die folgende iterative Formel beschreiben:

$$\vec{y}_{n+1} = \vec{y}_n + \frac{\Delta t}{6}\left(\vec{k}_1 + 2\vec{k}_2 + 2\vec{k}_3 + \vec{k}_4\right), \tag{41}$$

wobei die Zwischenwerte $\vec{k}_i$ durch:

$$\vec{k}_1 = f(t_n, \vec{y}_n), \tag{42}$$

$$\vec{k}_2 = f\left(t_n + \frac{\Delta t}{2}, \vec{y}_n + \frac{\Delta t}{2}\vec{k}_1\right), \tag{43}$$

$$\vec{k}_3 = f\left(t_n + \frac{\Delta t}{2}, \vec{y}_n + \frac{\Delta t}{2}\vec{k}_2\right), \tag{44}$$

$$\vec{k}_4 = f(t_n + \Delta t, \vec{y}_n + \Delta t\vec{k}_3) \tag{45}$$

definiert sind. Hierbei beschreibt f das zugrunde liegende Differentialgleichungssystem.

Das RK4-Verfahren ist besonders geeignet, um die Dynamik geladener Teilchen zu simulieren, da es die Stabilität und Genauigkeit auch bei kleinen Zeitschritten gewährleistet. Dies ist entscheidend für die Lösung der Bewegungsgleichungen im Magnetfeld, da $\vec{B}$ inhomogen ist und von der Position $\vec{s}(t)$ abhängt.

5.1 Abgestossenes Elektron

Im Skript `5_RugneKutta_HomCoil.ipynb` ist das Runge-Kutta-Verfahren (RKV) implementiert, um die Bahn eines Elektrons zu berechnen. Mit einem iterativen Algorithmus wurde untersucht, mit welcher z-positiven Anfangsgeschwindigkeit (v_{z0}) ein Elektron von einer Startposition s_0 genau die XY-Ebene

erreicht, also bei $z = 0$, und aufgrund des Einflusses des wirkenden Magnetfelds die Richtung ändert und wieder abgestossen wird.

In Abbildung 5 wird schematisch dargestellt, wie der Algorithmus aufgebaut wurde.

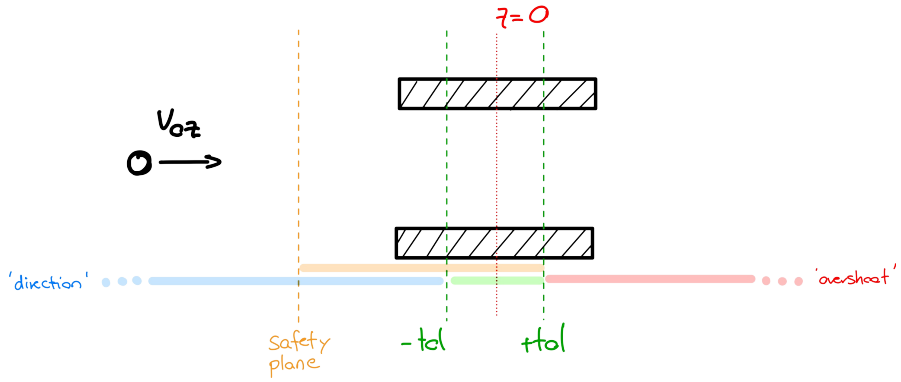


Abbildung 5: Schematische Darstellung des iterativen Algorithmus.

Um die richtige Geschwindigkeit zu finden, wird eine obere und eine untere Grenze angegeben: `v0_min` und `v0_max`. Das Runge-Kutta-Verfahren berechnet iterativ zu einem definierten Zeitschritt dt die Position und Geschwindigkeit des Elektrons. Eine akzeptable Toleranz `tolerance_z` wird angegeben. Somit erreicht ein Elektron $z = 0$, wenn es sich im Bereich $z = 0 \pm$ `tolerance_z` befindet. Folgende Punkte werden überwacht:

- Falls die z-Geschwindigkeit ihr Vorzeichen ändert, bedeutet dies, dass das Elektron die Richtung wechselt. Wenn dies geschieht, bevor $z = -$`tolerance_z` erreicht wird, wird die Simulation mit der aktuellen Geschwindigkeit abgebrochen, und ein `direction`-Flag wird gesetzt. Die simulierte Geschwindigkeit wird in der nächsten Iteration als untere Grenze verwendet. (Blauer Bereich in Abbildung 5)

- Falls die z-Position in der Simulation $z = +$`tolerance_z` überschreitet, bedeutet dies, dass die Geschwindigkeit des Elektrons zu hoch war. Folglich wird die Simulation abgebrochen, ein `overshoot`-Flag gesetzt, und die Anfangsgeschwindigkeit wird als obere Grenze verwendet. (Roter Bereich in Abbildung 5)

- Wenn der Toleranzbereich in einem Zeitschritt erreicht wird (Grüner Bereich in Abbildung 5), wird ein feinerer Zeitschritt genutzt, um das Verhalten des Elektrons genauer zu überprüfen. Falls während dieser Sicherheitsprüfung das Elektron dennoch die Toleranz überschreitet, wird es weiterhin als `overshoot` gekennzeichnet, und der Geschwindigkeitsbereich wird angepasst. Falls das Elektron jedoch zurückkehrt und seine z-Position einen definierten Schwellenwert (Safety plane in Abbildung 5) erreicht, wird die Simulation abgebrochen, und ein `success`-Flag wird gesetzt. (Oranger Bereich in Abbildung 5)

Wenn der Algorithmus mit einem `success`-Flag beendet wird, bedeutet dies, dass die gesuchte Geschwindigkeit gefunden wurde. Um den Geschwindigkeitsbereich zu verkleinern, wird ein einfacher Binary-Search-Algorithmus verwendet. Zusätzlich wurde eine dynamische Zeitschrittanpassung implementiert, die den Zeitschritt verkleinert, um möglichst präzise die Bahn des Elektrons untersuchen zu können.

Simulation

In diesem Abschnitt werden die Resultate der erfolgreichen Simulation und Bestimmung der gesuchten Geschwindigkeit präsentiert. Folgende Parameter wurden gesetzt:

$$s_0 = (0, 0.007, -0.05)$$
$$\texttt{v0_min} = \texttt{1e7}$$
$$\texttt{v0_max} = \texttt{2e8}$$
$$\texttt{tolerance_z} = \texttt{1e-6}$$

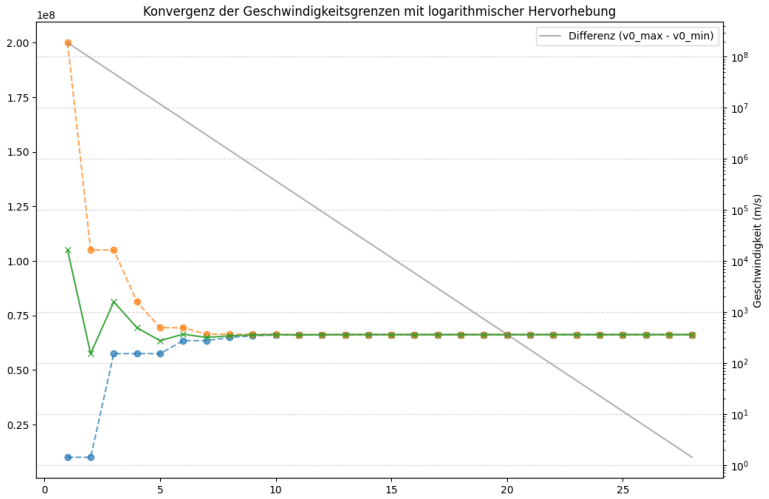


Abbildung 6: Visualisierung des Binary-Search-Algorithmus.

Abbildung 6 visualisiert das Verhalten des Binary-Search-Algorithmus und die daraus resultierende Konvergenz der Geschwindigkeit.

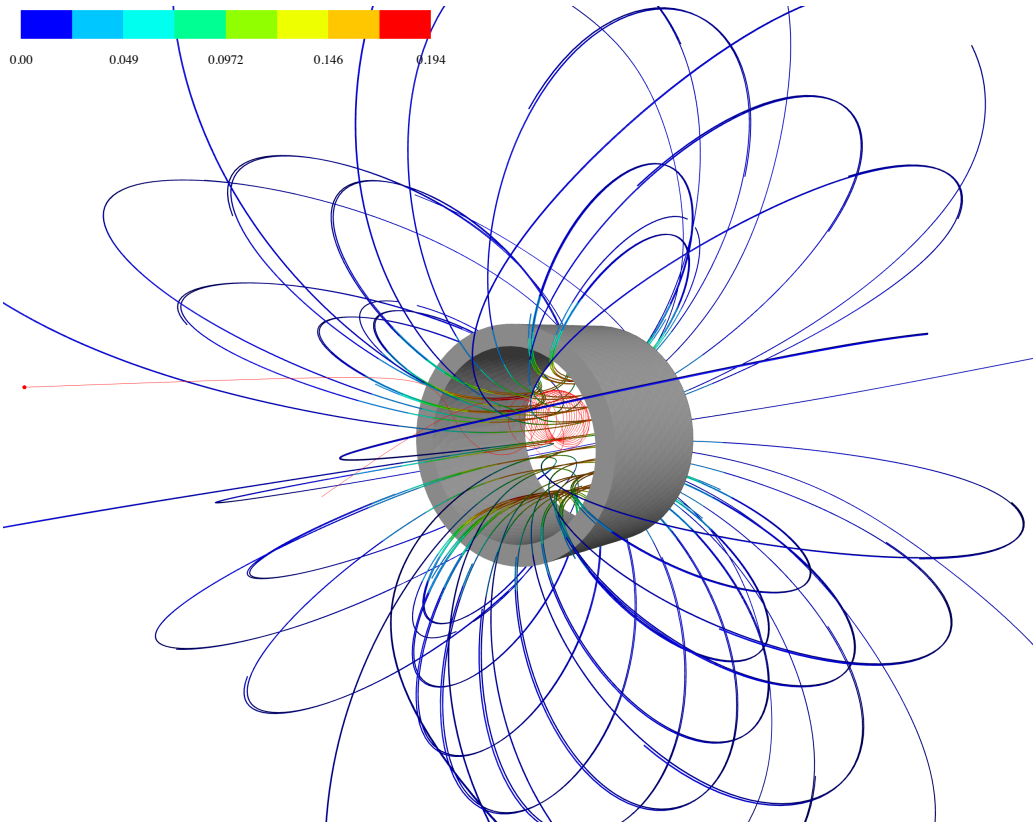


Abbildung 7: Visualisierung der Elektron Startposition (roter Punkt) und dessen Bahn (rot Linie) mit Feldlinien des Magnetfeldes B und die Homogenisierte Spule.

In Abbildung 7 ist die Bahn des Elektrons ersichtlich, die mithilfe des RKV simuliert wurde. Dank der Simulation wurde die nötige Anfangsgeschwindigkeit gefunden, die erforderlich ist, damit das Elektron die XY-Ebene innerhalb des Toleranzbereichs erreicht und vom Magnetfeld zurückgestossen wird. Von der Startposition $s_0 = (0, 0.007, -0.05)$ beträgt die gefundene Anfangsgeschwindigkeit: $v_{z0} = 66\,202\,467.941\,\mathrm{m/s}$.

5.2 Proton auf Kreisbahn

In diesem Abschnitt wurde das Ziel gesetzt, die Bahn eines Protons zu untersuchen (Code ersichtlich in `5_Proton_HomCoil.ipynb`). Im Detail soll das Verhalten eines Protons im Magnetfeld der homogenisierten Geometrie näher untersucht werden, um eine Anfangsposition und Anfangsgeschwindigkeit zu finden, bei der sich das Proton möglichst lange auf einer Kreisbahn bewegt.

Aus dem Zusammenhang, dass sich auf einer Kreisbahn die Zentripetalkraft und die Lorentzkraft des Protons ausgleichen müssen, folgt, dass der Radius mit der Formel

$$r = \frac{m \cdot v}{q \cdot B(r)} \tag{46}$$

abgeschätzt wird. Da das Magnetfeld B vom Radius r abhängt, kann dieser rekursiv analytisch bestimmt werden.

Simulation

In der Simulation ist die Startposition des Protons an den Koordinaten $s_0 = (r_0, 0, 0)$ mit einer Anfangsgeschwindigkeit von $10^5\,\mathrm{m/s}$ in positive y-Richtung. Der Anfangsradius $r_{\text{start}} = 0.004\,\mathrm{m}$ konvergiert bei der rekursiven Formel nach 7 Iterationen zu $6.091178 \cdot 10^{-3}\,\mathrm{m}$. Danach wird die Bahn simuliert. Abbildung 8 stellt den 2D-Plot der Bahn des Protons dar.

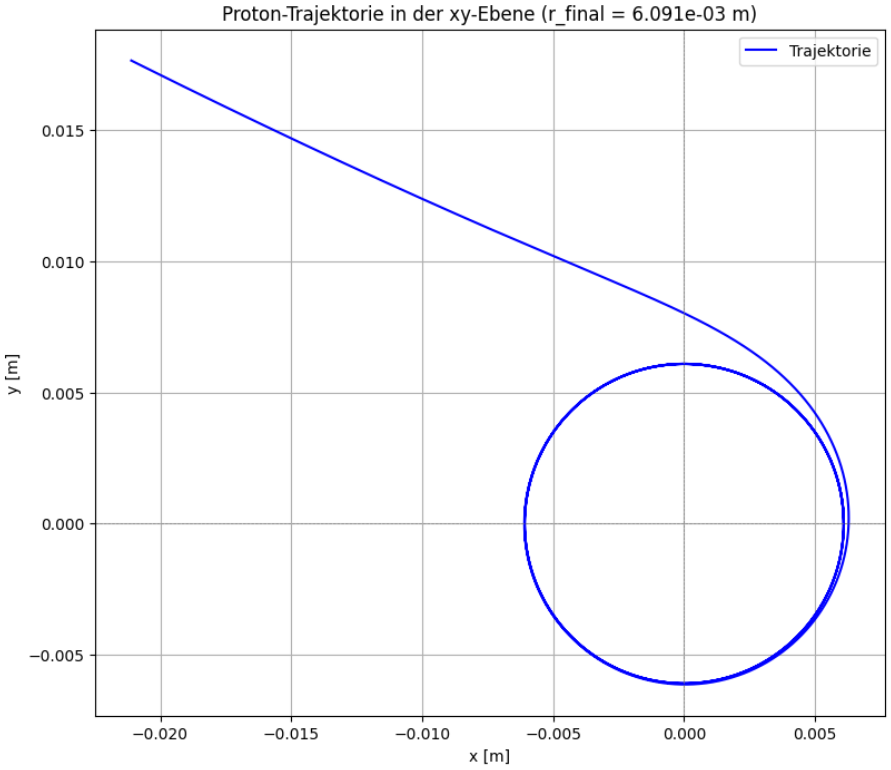


Abbildung 8: Visualisierung der Kreisförmige Proton-bahn.

In Abbildung 9 ist ersichtlich, wie stark der Einfluss des Magnetfeldes B auf das Proton wirkt und dieses schon nach zwei etwa Kreisförmige bahnen stark ablenkt wird.

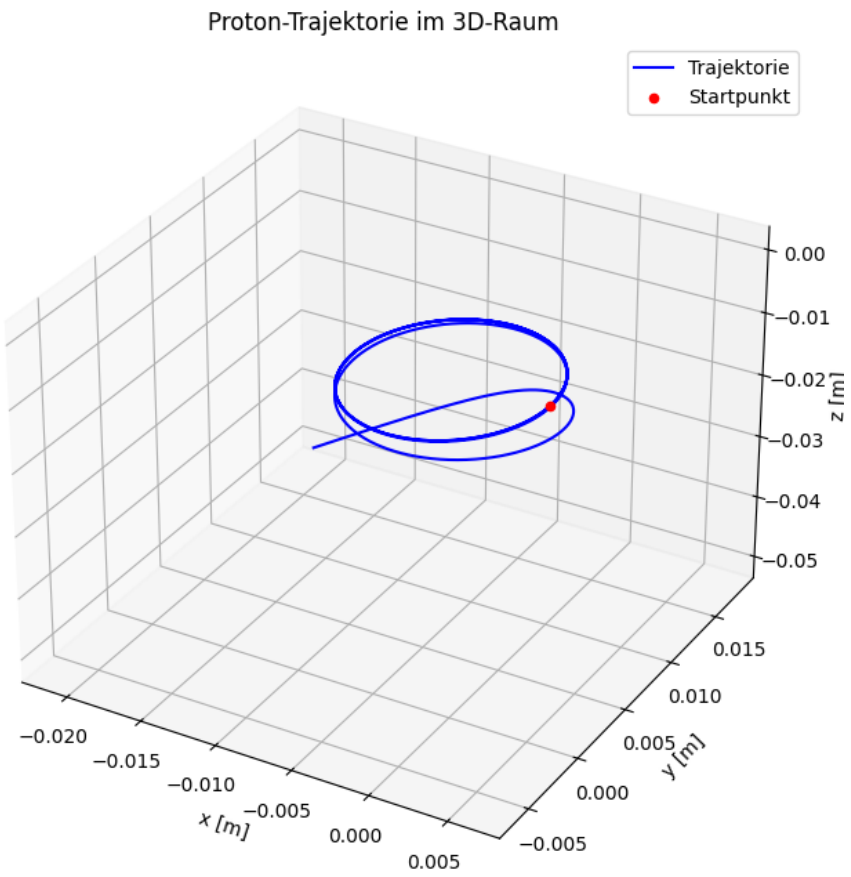


Abbildung 9: 3D-Visualisierung der Kreisförmige Protonbahn.

6 Implementierung und Training eines Physik-informierten Neuronalen Netzwerks (PINN)

6.0.1 Problemstellung

Wir beabsichtigen, die Bewegungsgleichungen für ein geladenes Teilchen (Elektron) zu lösen, das der Lorentzkraft in einem Magnetfeld $\mathbf{B}$ unterliegt. Damit soll das gleiche Verhalten wie in Aufgabe 5.1 gelernt werden. In klassischer Form lautet die Bewegungsgleichung:

$$m \frac{d^2\mathbf{r}(t)}{dt^2} = q\,\mathbf{v}(t) \times \mathbf{B}\big(\mathbf{r}(t)\big), \tag{47}$$

wobei

- m die Masse des Teilchens ist (z.B. die Elektronenmasse),
- q die Ladung des Teilchens ist,
- $\mathbf{r}(t)$ die Position des Teilchens bezeichnet,
- $\mathbf{v}(t) = \frac{d\mathbf{r}(t)}{dt}$ die Geschwindigkeit darstellt, und
- $\mathbf{B}\big(\mathbf{r}(t)\big)$ das Magnetfeld am Ort des Teilchens ist.

Die genauere Herleitung ist in Kapitel 2.3 zu finden. Wir versuchen damit nachzustellen ob wir auch ohne Runge-Kutte verfahren bei gebenener Geschwindigkeit ein Elektron dazu bringen können auf der Z achse im Magnetfeld umzukehren. In diesem Kontext verwenden wir ein **Physik-informiertes Neuronales Netzwerk (PINN)**, um $\mathbf{r}(t)$ zu approximieren. PINNs binden physikalische Gesetze in die Loss-Funktion während des Trainings ein, was den Bedarf an grossen gelabelten Datensätzen verringert und dem Modell ermöglicht, physikalisch konsistente Lösungen zu lernen.

6.0.2 Warum PINNs?

Traditionelle datengetriebene Methoden stossen oft an ihre Grenzen, wenn nur wenige Daten vorliegen oder die zugrundeliegende Physik komplex ist. PINNs umgehen diese Probleme, indem sie:

- Theoretische Modelle (die Lorentzkraftgleichung),
- Automatische Differentiation zur Berechnung von Ableitungen, die für das Residuum der zugehörigen Differentialgleichungen benötigt werden,
- Empirische oder synthetische Daten (hier: Anfangsbedingungen)

kombinieren.

Dies ermöglicht es dem Netzwerk, Lösungen robuster und realistischer zu erlernen, insbesondere wenn nur wenige oder schwer zu erzeugende Echtdaten verfügbar sind.

6.1 Netzwerkarchitektur

6.1.1 Wahl der Architektur

Für viele PINN-Anwendungen reicht ein vollständig verbundenes Feedforward-Netzwerk (FNN) in der Regel aus. In dem beigefügten Code besteht die Netzwerkklasse `TrajectoryPINN` aus:

- Einer Eingabeschicht, die die Zeit t als Skalar entgegennimmt,
- Einer verborgenen Schichten mit 128 Neuronen pro Schicht,
- Nichtlinearen Aktivierungsfunktionen`LeakyReLU`
- Einer Ausgabeschicht, die die Koordinaten $\big[x(t), y(t), z(t)\big]$ zurückgibt.

Die Aktivierungsfunktion wurde gewählt um Vanishing Gradients vorzubeugen.

6.1.2 Anpassungen für physikalische Randbedingungen

Um Anfangs- oder Randbedingungen in das Netzwerk zu integrieren, kann man:

- Gewisse Rand-/Anfangsbedingungen hart in die Architektur einkodieren (z.B. mittels Parametrisierungen),
- Randbedingungen über zusätzliche Terme in der Loss-Funktion erzwingen (der hier verwendete Ansatz).

In unserem Code werden die Anfangsbedingungen über *Anfangsbedingungs-Losses* hinzugefügt. Lösungen, die vom bekannten Anfangszustand abweichen, werden dadurch entsprechend bestraft (penalized). Die Anfangsbedingungen werden durch die anfängliche Teilchenposition $\mathbf{r}_0 = (0.0, 0.007, -0.05)$ sowie die Anfangsgeschwindigkeit $\mathbf{v}_0 = (0.0, 0.0, 1.21379551 \times 10^8)$ festgelegt. Auf diese Weise wird sichergestellt, dass das PINN die korrekte Startlage und Startgeschwindigkeit im Training berücksichtigt. Die Anfangsgeschwindigkeit entspricht dem Wert, den wir mit unserem Algorithmus bestimmt haben.

6.2 Design der Loss-Funktion

6.2.1 Residuum aus den grundlegenden Bewegungsgleichungen

Das zentrale physikalische Gesetz hier ist die Lorentzkraftgleichung, die in Bezug auf die Beschleunigung $\mathbf{a}(t) = \frac{d\mathbf{v}(t)}{dt}$ umgeschrieben werden kann:

$$m\,\mathbf{a}(t) \;-\; q\,\mathbf{v}(t) \times \mathbf{B}\big(\mathbf{r}(t)\big) \;=\; 0. \tag{48}$$

Um dies im PINN einzubetten:

1. Wird automatische Differentiation verwendet, um die erste Ableitung $\mathbf{v}(t) = d\mathbf{r}(t)/dt$ und die zweite Ableitung $\mathbf{a}(t) = d^2\mathbf{r}(t)/dt^2$ zu berechnen.
2. $\mathbf{B}\big(\mathbf{r}(t)\big)$ wird durch Abfragen eines NGSOLVE-Mesh-basierten Interpolationsnetzes ermittelt.

3. Anschliessend wird das Residuum $\mathbf{F}_{\text{res}} = m\mathbf{a} - q\,\mathbf{v} \times \mathbf{B}$ gebildet und dessen Norm berechnet, z.B. $\|\mathbf{F}_{\text{res}}\|^2$.

6.2.2 Anfangsbedingungs-Loss

Die vom Modell vorhergesagte Lösung muss die Anfangsbedingungen für die Position $\mathbf{r}(0)$ und Geschwindigkeit $\mathbf{v}(0)$ erfüllen. Im Code werden diese Bedingungen als mittlerer quadratischer Fehler ausgedrückt:

$$\mathcal{L}_{\text{IC}} = \|\mathbf{r}(0) - \mathbf{r}_0\|^2 + \|\mathbf{v}(0) - \mathbf{v}_0\|^2,$$

wobei $\mathbf{r}_0$ und $\mathbf{v}_0$ die bekannten Anfangspositionen bzw. -geschwindigkeiten sind. Dieser Term wird der Gesamtloss-Funktion hinzugefügt.

6.2.3 Gesamtloss

Die Kombination aus physikalischem Residuum und Anfangsbedingungen ergibt:

$$\mathcal{L}_{\text{total}} \;=\; \mathcal{L}_{\text{physics}} \;+\; \mathcal{L}_{\text{IC}}.$$

Mitunter werden dynamische oder weiter verfeinerte Gewichtungsstrategien angewandt. In unserem dynamischen Gewichtungsansatz wird zunächst die Gesamtverlustfunktion `total_loss` als Summe aus dem physikalischen Verlust (`physics_loss_val`) und dem Anfangsbedingungsverlust (`ic_loss`) ermittelt. Anschliessend wird für jede Teilkomponente ein Gewicht berechnet, das vom Verhältnis der Gesamtverlustfunktion zur doppelten Grösse des jeweiligen Verlustes abhängt. Beispielsweise entsteht das Gewicht für die physikalische Komponente über

$$\texttt{physics_weight} \;=\; \frac{\texttt{total_loss}}{2\left(\texttt{physics_loss_val} + 10^{-8}\right)}.$$

Analog wird das Gewicht der Anfangsbedingungskomponente `ic_weight` definiert. Schliesslich wird der gewichtete Gesamtverlust `weighted_loss` durch Multiplikation der jeweiligen Teilverluste mit ihren Gewichten und anschliessender Aufsummierung bestimmt. Das verhindert das sich die einzelnen losses zu weit voneinander entfernen und somit kaum mehr lernbar werden.

6.3 Datenaufbereitung und Normalisierung

6.3.1 Feldinterpolation auf Basis eines Meshs

In vielen praktischen Fällen wird das Magnetfeld $\mathbf{B}$ aus numerischen Simulationsdaten (z.B. Finite-Elemente-Gitter) oder Messungen gewonnen. Unser code nutzt `netgen` und `ngsolve` für die Definitionen auf Basis unseres FEM Gittermodells,

6.3.2 Grenzen und Clamping des Definitionsbereichs

Da $\mathbf{B}$ nur auf einem diskreten Mesh (in einem bestimmten Bereich definiert ist, prüft der Code, ob sich die Teilchenposition $\mathbf{r}(t)$ innerhalb des zulässigen Gebiets befindet. Falls nicht, wird $\mathbf{B} = \mathbf{0}$ zurückgegeben. Das Magnetfeld ist an diesen Punkten also Null.

6.4 Trainingsprozess

6.4.1 Wahl des Optimierungsverfahrens

Wir haben einen hybriden Ansatz für die Optimierung des PINN implementiert. Konkret kamen folgende Methoden zum Einsatz:

- **Adam**: Ein adaptiver Gradienten-Algorithmus, der die Vorteile von Momentum und adaptiven Lernraten kombiniert. Er ist besonders effektiv bei Problemen mit spärlichen Gradienten und kann in frühen Trainingsphasen die Konvergenz beschleunigen. Wir verwendeten eine anfängliche Lernrate von $\texttt{lr} = 0.1$, die durch einen Learning-Rate-Scheduler nach und nach reduziert wurde, um die Stabilität in späteren Trainingsstadien zu gewährleisten.
- **AdamW**: Eine Variante von Adam, welche Weight Decay als separaten Regularisierungsterm behandelt, anstatt ihn mit dem Gradienten-Update zu kombinieren. Dies verbessert die Generalisierung, insbesondere bei Netzwerken mit grossen Parameterumfängen. Analog zu Adam starteten wir mit einer Lernrate von $\texttt{lr} = 1.2 \times 10^{-3}$ und setzten ebenfalls einen Scheduler zur schrittweisen Reduktion ein.

6.4.2 Lernraten-Schedules

Ein `ReduceLROnPlateau`-Scheduler wird eingesetzt, um die Lernrate automatisch zu reduzieren, wenn die Loss-Funktion stagniert. Dies hilft , lokale Minima oder langsame Regionen in der Fehlerlandschaft zu vermeiden.

6.4.3 Konvergenzkriterien

Unsere Kriterien sind:

- Erreichen eines ausreichend kleinen Werts von $\mathcal{L}_{\text{total}}$,
- Erreichen einer maximalen Epochenzahl.

Im Code stoppt das Training, wenn die Loss-Funktion unter den Schwellenwert $\approx 10^{-6}$ fällt oder nach einer im Code definierten Anzahl von Epochen.

6.5 Lernkurve

Unser Model wurde mit einem Hybrid von ADAM und ADAMW Vor-trainiert um denn Loss zu senken und anschliessend zwei mal mit ADAMW nach-trainiert. Insgesamt wurde über 1'300'000 Epochen trainiert. Wie für PINN's typisch konvergierte der Loss nur langsam und fluktuierte Stark was in Abbildung 10 und 11 ersichtlich ist. Durch ein PC Absturz ging die erste Lernkurve verloren, der Loss wurde dabei von $2.4555 \cdot 10^{16}$ auf ≈ 24 reduziert.

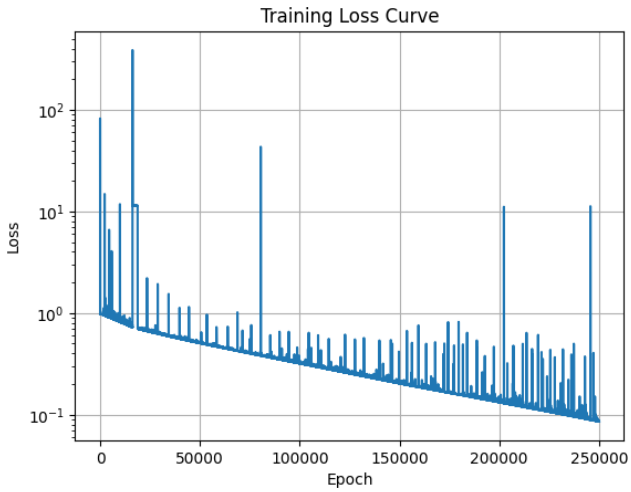


Abbildung 10: Zweite Lernkurve

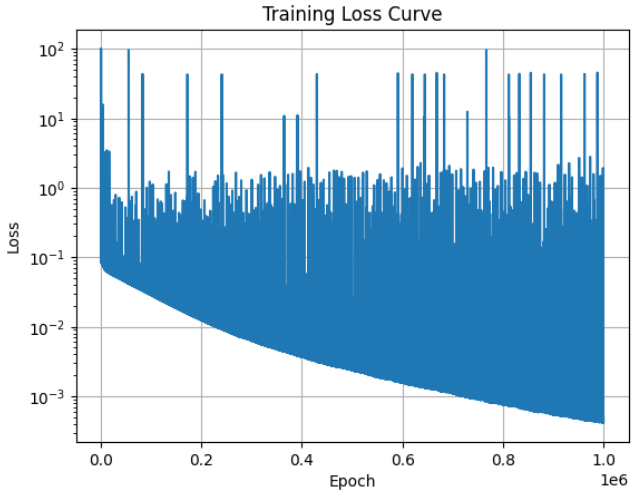


Abbildung 11: Dritte Lernkurve

6.6 Elektron Trajektorie

Die in Abbildung 12 und 13 sichtbaren Elektronenspuren zeigen das dass verhalten des Elektrons wie in Kapitel 5.1 nicht erfolgreich gelernt werden konnte. Das Elektron konnte nicht erfolgreich zur Umkehr gebracht werden. Die Position des Elektron wurde für den Zeitrum $t \in [0{,}1 \cdot 10^{-5}]$ vorhergesagt. Interessanterweise scheint die Oszilation insbesondere auf einer 45 Grad Achse zwischen der X und Y Achse stattzufinden.

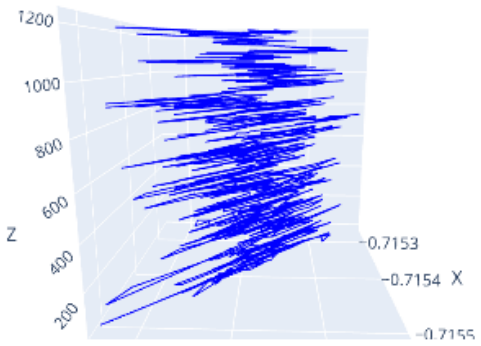


Abbildung 12: Flugbahn des Elektrons im Magnetfeld, sicht von der Y Achse

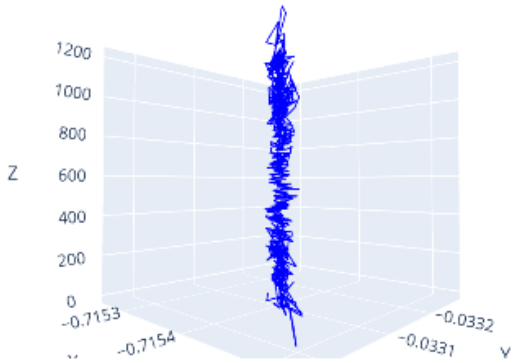


Abbildung 13: Flugbahn des Elektrons im Magnetfeld, Sicht aus einem 45 Grad Winkel zwischen X und Y Achse.

6.7 Validierung und Test

Wir wollten die Ergebnisse mit unserer NGSolve Lösung als Referenztrajektorie vergleichen. Rein visuell war allerdings bereits klar das unsere PINN Lösung zu stark abweicht vom Runge Kutta Original und dementsprechend ein Vergleich bzw. ein Fehler kaum Aussagekraft hätte.

6.8 Mögliche Erweiterungen

Zur Verbesserung des Trainings könnten physikalische Constraints mit Datenpunkten aus einer Referenzsimulation kombiniert werden, sodass das PINN zusätzlich an konkreten Trajektorien orientiert wird. Eine optimierte Netzwerkarchitektur mit erweiterten Schichten, alternativen Aktivierungsfunktionen wie Fourier-Features und Regulierungstechniken (z. B. Weight Decay, Dropout) könnte helfen, hochfrequente Bewegungen besser zu erfassen und Overfitting zu reduzieren. Ein adaptives Verfahren zur Auswahl von Collocation Points könnte die Trainingsressourcen effizienter nutzen, indem Stützstellen gezielt in Bereichen starker Gradienten verfeinert werden. Fortgeschrittene Optimierungsstrategien wie LBFGS, SGD mit Momentum oder Differential Evolution sowie Preconditioning könnten die Stabilität und Konvergenz des Trainings weiter verbessern. Ein stufenweiser Lernansatz (*Curriculum Learning*), bei dem das Netzwerk zunächst in vereinfachten Szenarien trainiert wird, würde extreme Schwankungen im Loss minimieren. Schliesslich könnte eine feinere oder adaptive Mesh-Definition mit `netgen` und `ngsolve` unphysikalisches Verhalten durch verbesserte Magnetfeld-Interpolation verhindern.

6.9 Fazit

Obwohl das PINN in dieser Arbeit nicht das gewünschte Ergebnis lieferte, bleibt die Methodik potentiell vielversprechend, um physikalische Systeme auch mit wenigen Daten zu approximieren. Entscheidend für den Erfolg ist jedoch eine sorgfältige Kontrolle der Hyperparameter, eine adäquate Netzwerkarchitektur und die Berücksichtigung sinnvoller Trainingsstrategien (z. B. Data-Loss, adaptives Collocation und stufenweises Lernen). In zukünftigen Arbeiten könnten solche Ansätze ein stabileres und genaueres PINN hervorbringen, das letztlich die angestrebte Umkehrbewegung des Elektrons erfolgreich nachbildet. Gleichzeitig ist es jedoch wichtig, Aufwand und Nutzen sorgfältig abzuwägen, da etablierte numerische Verfahren in spezifische Fällen weiterhin die effizientere Wahl darstellen können.

7 Diskussion

Mit der Durchführung dieses Projekts konnte festgestellt werden, dass `ngsolve` ein sehr leistungsfähiges Tool für die Lösung numerischer Aufgaben ist. Insbesondere wurde gezeigt, dass die Simulation des Magnetfeldes in einer Spule mit homogenisierter Geometrie hinsichtlich der Rechenzeit vorteilhaft sein kann. In unseren Versuchen war diese Methode etwa 90 % schneller (siehe Tabelle 1). Der resultierende Unterschied zwischen inhomogener und homogener Geometrie, wie bereits in Kapitel 4.3 erwähnt, beträgt: $\mu \pm \sigma = -0.000215 \pm 0.001415\,\mathrm{T}$.

Der negative Wert deutet darauf hin, dass das homogene Feld eine Tendenz hat, leicht grössere Werte zu berechnen als das inhomogene Feld. Es muss daher bewusst sein, dass die Verwendung der homogenen Simulation für die Dimensionierung von Systemen je nach Komplexität zu einer leichten Überdimensionierung führen kann.

Die Standardabweichung ist im Vergleich zum Mittelwert relativ gross, was darauf hinweist, dass eine gewisse Unsicherheit besteht. Eine mögliche Erklärung ist, dass in der gesamten Geometrie grosse Feldunterschiede auftreten und die Betrachtung der gesamten simulierten Geometrie daher nicht vorteilhaft ist. Wenn präzisere Simulationen erforderlich sind, sollte untersucht werden, wie die Genauigkeit der homogenisierten Geometrie weiter verbessert werden kann.

Bei der Simulation der Bahn des Elektrons in Kapitel 5 wurde anfänglich ein Magnetfeld basierend auf einem $H(\mathrm{curl})$-Raum der Ordnung 2 verwendet. Eine höhere Ordnung bedeutet, dass komplexere und feinere Variationen des Magnetfelds besser erfasst werden können.

Die Ordnung des Raumes wurde später erhöht, was eine deutlich bessere Genauigkeit der Simulationsergebnisse ermöglichte. In den ersten Durchläufen konnte die Simulation nur mit einer Toleranz von `tolerance_z` $= \pm 1 \cdot 10^{-3}\,\mathrm{m}$ erfolgreich durchgeführt werden. Betrachtet man die Geometriemasse, so stellt sich heraus, dass diese Toleranz sehr grob ist. Mit der Verfeinerung des Magnetfelds (Erhöhung der `order`) konnte schliesslich eine erfolgreiche Simulation mit einer Toleranz von `tolerance_z` $= \pm 1 \cdot 10^{-6}\,\mathrm{m}$ durchgeführt werden.

Die in Kapitel 6.6 vorgestellten Ergebnisse zeigen, dass das trainierte PINN die gewünschte Elektronentrajektorie – insbesondere die Umkehrbewegung – nicht akkurat abbilden konnte. Dieses Scheitern hat mehrere potenzielle Ursachen.

Erstens ist die Bewegung eines Elektrons in einem dreidimensionalen Magnetfeld äusserst empfindlich gegenüber Parametervariationen. Schon geringe Abweichungen in den Anfangsbedingungen, insbesondere bei der Startgeschwindigkeit, können zu einem vollständig anderen Verhalten führen.

Zweitens ist das Training von PINNs generell herausfordernd, da eine Vielzahl an Hyperparametern (z. B. Lernraten, Netzwerkarchitektur und Gewichtungsfaktoren der Teilverluste) aufeinander abgestimmt werden müssen. Eine unzureichend justierte Loss-Gewichtung kann leicht dazu führen, dass das PINN in lokalen Minima steckenbleibt.

Drittens kann die diskrete Mesh-Definition des Magnetfelds zu unphysikalischem Verhalten führen. Schliesslich fehlt bei rein physikbasierten PINNs oft ein Datenanker, der ein Data-Loss aus expliziten Trajektorienpunkten liefern könnte.

Literatur

[1] M. Saxer and S. Bortolato, "Hana pde modeling fem," Januar 2025. URL: https://github.com/Saxermi/hana_pde_modeling_fem [12.01.2025].

[2] S. Stingelin, "Hana numerik sep projekt," Oktober 2024. URL: https://moodle.zhaw.ch/mod/assign/view.php?id=1448382 [30.10.2024].